Cluster Computing
https://doi.org/10.1007/s10586-023-04228-y

Cdascaler: a cost-effective dynamic autoscaling approach for containerized microservices

Numan Shafi[1] · Muhammad Abdullah[1] · Waheed Iqbal[1] · Abdelkarim Erradi[2] · Faisal Bukhari[1]

Received: 7 August 2023 / Revised: 27 November 2023 / Accepted: 29 November 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract
Microservices are containerized, loosely coupled, interactive smaller units of the application that can be deployed, reused, and maintained independently. In a microservices-based application, allocating the right computing resources for each containerized microservice is important to meet the specific performance requirements while minimizing the infrastructure cost. Microservices-based applications are easy to scale automatically based on incoming workload and resource demand automatically. However, it is challenging to identify the right amount of resources for containers hosting microservices and then allocate them dynamically during the auto-scaling. Existing auto-scaling solutions for microservices focus on identifying the appropriate time and number of containers to be added/removed dynamically for an application. However, they do not address the issue of selecting the right amount of resources, such as CPU cores, for individual containers during each scaling event. This paper presents a novel approach to dynamically allocate the CPU resources to the containerized microservice during the autoscaling events. Our proposed approach is based on the machine learning method, which can identify the right amount of CPU resources for each container, dynamically spawning for the microservices over time to satisfy the application's response time requirements. The proposed solution is evaluated using a benchmark microservices-based application based on real-world workloads on the Kubernetes cluster. The experimental results show that the proposed solution outperforms by yielding a 40% to 60% reduction in violating the response time requirements with 0.5× to 1.5× less cost compared to the state-of-art baseline methods.

Keywords Microservices · Web application autoscaling · Kubernetes · CPU cores · Horizontal scaling · Hybrid · Cost effective

1 Introduction

Cloud computing offers on-demand resources and services, including computing, networking, storage, databases, intelligence, and analytics [1–3]. It decreases the operational cost, runs the application efficiently, and scales the resources when needed. Also, users only pay for the resources when they actually use them. Dynamic resource provisioning and application management on cloud infrastructure is crucial and depends on the selected application architecture [4].

Nowadays, two commonly used architectures for building cloud-hosted applications are monolithic and microservice. An application built using monolithic architecture is packaged as a single unit containing all tiers, e.g., user interface, database, and business logic, and then deployed on a single computing resource (virtual or

✉ Muhammad Abdullah
muhammad.abdullah@pucit.edu.pk

Numan Shafi
phdcsf20m504@pucit.edu.pk

Waheed Iqbal
waheed.iqbal@pucit.edu.pk

Abdelkarim Erradi
erradi@qu.edu.qa

Faisal Bukhari
faisal.bukhari@pucit.edu.pk

[1] Faculty of Computing & IT, University of the Punjab, Lahore, Pakistan

[2] Department of Computer Science and Engineering, College of Engineering, Qatar University, Doha, Qatar

Published online: 18 January 2024

physical machine). In contrast, applications built using microservice architecture consist of multiple independent, loosely coupled interactive small components. This allows individual components to deploy, scale, and maintain on different computing resources. Microservice-based applications incur 70% less cost than monolithic applications [5]. This is mainly due to the appropriate resources being allocated to each component and only scaling those components requiring more resources based on the incoming workload. Whereas for a monolithic-based application hosted on a single computing instance, the entire application is scaled whenever the performance is degraded. This leads to the over-provisioning of computing resources allocated to the monolithic application. However, it is challenging to efficiently scale the microservice individual components by detecting and scaling the components contributing to degrading the performance of the application.

Cloud provides virtual computing resources to maximize the utilization of the hardware infrastructure. Traditionally Virtual Machines (VMs) are provisioned for the end users to host their applications. However, containers have recently been introduced, which are lightweight compared to VMs and provide similar functionalities for ensuring application isolation [6, 7]. Microservices are containerized, loosely coupled, interactive smaller units of the application. These small independent units (services) of the application can be deployed, reused, and maintained in separate containers. This allows for dynamically managing the computing resources for individual services to achieve seamless scalability of the applications easier than traditional monolithic applications. Containers are appealing for deploying and managing microservices-based applications using some orchestration platform, for example, Kubernetes [8], which can help to manage microservices easily compared to using traditional VMs.

For microservices-based applications, allocating the right computing resources to each microservice is important to meet the specific performance requirements while ensuring cost-effectiveness. For any application, if allocated computing resources are under-provisioned, it can yield performance issues, while over-provisioned resources can incur a high operational cost. Therefore, dynamic resource allocation is required to manage application resources effectively. Each microservice may have different resource demands; for example, some may require heavy memory and IO resources, while others may demand heavy CPU resources. Identifying the resource demands of individual microservices and then dynamically allocating appropriate resources is important and challenging [9, 10].

There are two popular strategies for dynamic resource allocation: reactive and predictive autoscaling methods [11–13]. Reactive autoscaling makes scaling decisions based on heuristics using specific metrics such as CPU utilization, response time, memory usage, and IO usage. This method adds more resources to the application when the selected resource metric crosses a specific upper threshold and retracts resources when a lower threshold is reached. However, determining the correct threshold values and the optimal amount of resources to allocate or retract can be challenging. On the other hand, predictive autoscaling uses intelligent techniques, such as machine learning methods [14, 15], to proactively manage application resources. This approach can accurately predict resource demand and allocate resources accordingly, making it more efficient than reactive autoscaling. By leveraging historical usage patterns and trends, predictive autoscaling can optimize resource allocation and provide better application performance and cost savings.

Existing autoscaling solutions [16, 17] focus on identifying the appropriate time and number of VMs or container instances with the static amount of resources, such as CPU cores to be added or removed dynamically for an application. However, they do not address the dynamically selecting the required amount of resources for each container, such as CPU cores, during each scaling event. CPU cores refer to the physical or logical processing units (cores) within a computer or server's central processing unit (CPU). In containerized applications, CPU cores for individual containers refer to allocating or assigning CPU processing units (cores) to each container. To demonstrate the importance of allocating a different number of CPU cores to containers during autoscaling, we conducted a toy experiment on a cloud-native microservice benchmark application consisting of ten different services [18].

In the toy experiment, we performed two iterations; in the first iteration, we allocated two CPU cores to each of the ten microservices, resulting in a total of ten containers. In the second iteration, we allocated two containers to each microservice by assigning only one CPU core to each container, resulting in twenty containers. Both iterations had an equal number of CPU cores allocated to the application and used the same workload to evaluate the application's performance. Figure 1 shows the observed response times during the experiment for both iterations. The results show that containers with one CPU core performed better than those with two CPU cores in some intervals (e.g., during intervals 6, 16, 18, 20, 22, 30, 38, 52, 65, and 72), while containers with two CPU cores outperformed those with one CPU core in other intervals. These results highlight the importance of dynamically assigning CPU cores for each container during autoscaling, which can significantly help to improve the application performance.

In this article, we present a novel approach dynamically allocate the CPU resources to the containerized

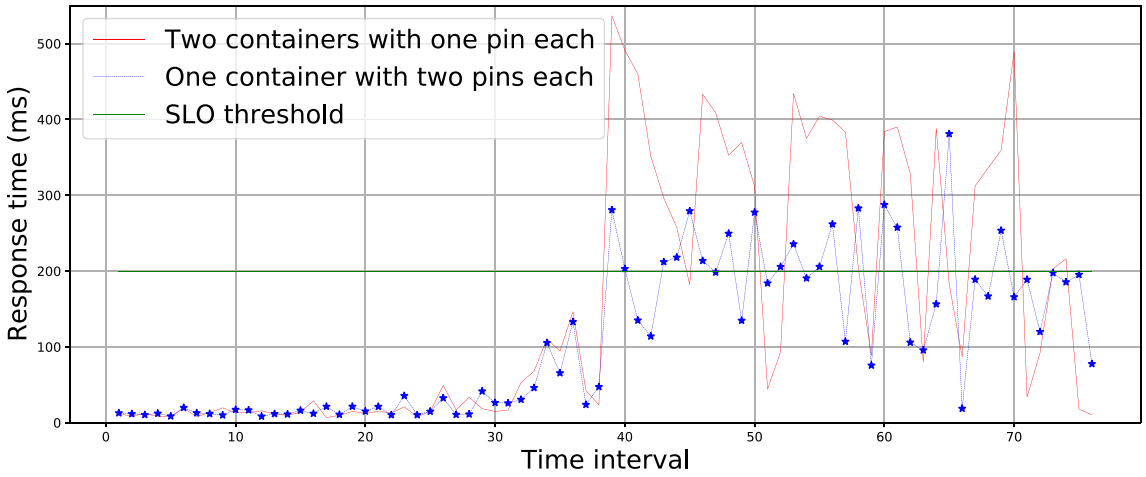


Fig. 1 Test experiment with two containers with one core and one container with two cores allocation

microservice during the autoscaling decisions. Our proposed approach is based on the machine learning method, which can identify the right amount of CPU resources for each container, dynamically spawning for the microservices over time to satisfy the application's response time requirements. The system collects performance traces for a target microservice application using different CPU configurations for each microservices while serving the incoming workload. The performance traces are used to prepare a dataset for training a model which can predict the CPU cores and the number of containers for each microservice, considering the forecasted workload and required response time. Finally, the learned policies are passed to the autoscaling controller, which scales in/out the cores and replicas. The proposed system offers a cost-effective and efficient solution to containerized microservices, improving performance while minimizing operational costs.

The main contributions of this paper are:

1. Investigate the impact of the CPU cores during the autoscaling for microservices-based applications.
2. Propose a novel, cost-effective, and dynamic autoscaling system named CDAScaler for containerized microservices.
3. Evaluate the proposed solution CDAScaler under different real-time workloads and compare it with state-of-the-art strategies.

The rest of the paper is organized as follows. Section 2 presents related work. Section 3 describes the major components of the CDAScaler. Section 4 discusses the experimental design for evaluating the CDAScaler. Section 5 presents the experimental results and evaluations. Conclusion and future work are discussed in Sect. 6.

2 Related work

Microservices are the smaller units of the application performing some functional requirements. These units can communicate with each other, can scale independently, and can be thrown out if required. Microservices can be written in a different language and use diverse data storage rather than strictly following the single language and single data storage for the whole application [19]. Microservices architecture is frequently used architecture in fast-growing companies, i.e., Netflix. This architecture is developed by taking inspiration from service-oriented computing [20]. Cloud computing provides distributed microservices architecture because microservices are cohesive, self-contained, and independent [21] [22]. Each cloud service provider (both public and private) is trying to give the facility of deploying and managing the microservices efficiently, as managing and resource provisioning is a critical and challenging task in cloud computing [23–27]. Kubernetes have simplified the configuration and management of the microservices. Moreover, Kubernetes is most widely used to orchestrate the microservices [28]. Significant progress has been made in implementing autoscaling for microservices through the use of Kubernetes and Docker Swarm, utilizing a variety of rule-based, proactive, and predictive strategies. These predictive techniques often incorporate machine learning, deep learning, and reinforcement learning to forecast resource requirements accurately [29–32]. Despite this ongoing advancement, resource provisioning remains a critical and challenging aspect of the process.

Using Kubernetes, rule-based autoscaling of the microservices is the easiest and most commonly used strategy. A lot of work is done on it. For example, Toka et al. [43] proposed HPA+ on Kubernetes for autoscaling highly varying incoming requests. They trained the

regression, unsupervised, deep learning, and reinforcement learning models to predict income request rates. Training many models takes more time and may cause service level agreement (SLA) violations. Ruiz et al. [37] paid more attention to infrastructure development that caused fewer violations. They came with on-premise Kubernetes infrastructure's autoscaling. They used rule-based autoscaling for microservices. They focused on the infrastructure-level implementation of automatic resource management; used CPU and memory usage as an evacuation matrix. Sachidananda et al. [40] proposed the COLA model for autoscaling. They gave the idea of autoscaling of dependent microservices. In COLA, they checked the most congested services, which use more CPU.

Some researchers used a proactive approach to scale in and scale out microservices on different benchmarks. For example, authors [34] used each request's latency for autoscaling sock shop and social network microservices. They built a dependency graph of each microservice and found the correlation based on the incoming load on each service. They calculated the latency ratio (p90/p50) and scaled them on a specified threshold. They used Bayesian optimization for how many pods should be scaled to meet the SLA. Ezzeddine et al. [41] discussed the autoscaling solution of event-driven microservices. They make sure of SLA in consumer microservice. They used the machine learning method, predicted the next arrival rate, and compared the consumption rate if arrival got greater than scaling up the service to meet the SLA agreement. Guangba et al. [34] proposed a microscaler that automatically identified the services that need to be scaled to fulfill the criteria of SLA (service level agreement) with optimal cost. The authors collected the quality of the service matrix in the proposed model through service mesh. Service mesh is configurable, secure, reliable, and observable. Then, with the help of service power, they highlighted the over-provisioning and under-provisioning services along with the service dependency graph. Highlighted services then passed through online and step-by-step heuristic approaches to give optimal scaled services.

Some authors used machine learning models that forecast required containers of the specific microservice and provide them before the SLA violations. For example, [16] used CPU utilization for autoscaling of fast Fourier transformation and temperature prediction microservices. Initially, they used reactive autoscaling to train the regression model. Based on provided data, they started to predict the following required containers to handle the upcoming requests. They used an online learning technique for prediction. The authors [36] proposed PEMA(Practical Efficient Microservice Autoscaling) for autoscaling multiple microservices. The authors used a rule-based technique to avoid the heavy training of ML models. Abdullah et al. [17] proposed the burst-aware autoscaling technique. In the proposed methodology, they trained their model on previous k example points, considering the response time and the number of requests. They also logged the real-time data. The authors used synthetic and real-world workloads. They also performed their experiment on a single service.

Deep learning models are frequently used nowadays. Some researchers used a deep learning model for resource provisioning of the microservices. For example, [35] proposed the HANSEL model for horizontal scaling of the microservices. They trained the model with synthetic data and predicted the load of microservices. They did not handle the jitter's overhead of workload. Razavi et al. [38] proposed an FA2 model to autoscale the deep learning interface serving system. FA2 is composed of a monitor, optimizer, and controller. The proposed model did not handle the fluctuating workload. It gave more SLA on changing workload. Khaleq et al. [39] experimented on Google's Kubernetes engine. They used the real-time workload of the Twitter disaster management system. Dang-Quang et al. [44] used bi-directional LSTM for autoscaling Kubernetes. The proposed model considers the HTTP request rate, but latency and CPU utilization are critical parameters.

Some researchers used reinforcement learning for resource provisioning and autoscaling of the microservices [45]. For example, the authors [33] used the adaptive threshold technique using reinforcement learning. They used both single and multi-agents for microservice autoscaling. In a single agent, both CPU and memory thresh are decided by the single agent, while in a multi-agent, CPU, and memory thresh are determined by a separate agent. The authors did not include some resource matrices such as CPU cores, I/O, and network rate. They also did not the comparison of SLA violations of their work with existing state-of-the-art work. Zafeiropoulos et al. [42] discussed autoscaling serverless functions using reinforcement learning. They used CPU usage, memory usage, CPU threshold, memory threshold, active serverless function, latency, and throughput for the RL environment, based on the reward and punishment proposed model to autoscale the serverless function without violating the SLA in terms of latency. Furthermore, Cheng et al. [46] explored real-time job scheduling in a hybrid cloud, emphasizing cost-efficient executions with minimal response time. This approach optimizes the costs by dynamically selecting suitable virtual machines for incoming jobs using deep reinforcement learning techniques. In contrast, the CDAScaler technique aims for cost-efficiency and optimal resource allocation based on the dynamic autoscaling of microservices. CDAScaler utilizes predictive modeling and dynamic resource allocation to optimize container count and pin allocation, ultimately

enhancing overall application performance and enabling efficient scaling decisions. Moreover, Cheng et al. [47] talk about making cloud computing work better for online businesses by scheduling tasks in an efficient way. They use a smart technique called deep reinforcement learning to improve the scheduling, focusing on saving money and ensuring quick user responses. On the other hand, the proposed CDAScaler technique is used for managing cloud services. It's not just about scheduling tasks quickly; it's about making sure containers and cores are used in the best way. CDAScaler aims for smarter and more efficient decision-making in managing the entire system to improve applications' performance.

We present a comprehensive comparison of CDAScaler with existing state-of-the-art auto-scaling strategies in Table 1. No existing study focuses on predicting the optimal number of CPU cores for each containerized microservice during horizontal autoscaling, a key challenge addressed by CDAScaler. Our approach incorporates dynamic resource allocation of CPU cores and container count, enabling optimal utilization of these resources and facilitating efficient scaling decisions at a granular level. Furthermore, CDAScaler efficiently handles the scaling of multiple interdependent microservices within an application-a critical challenge requiring a holistic approach to resource allocation across all services. This ensures optimal scaling decisions for individual microservices while considering the overall application's performance. Operating primarily in cloud environments, CDAScaler is designed to address the specific needs of cloud-hosted applications, ensuring cost-effectiveness and enhanced performance through efficient microservices scaling. Additionally, our strategy employs a proactive approach during autoscaling, intelligently identifying the required number of CPU cores and containers for each microservice to handle incoming workloads, thereby enhancing application responsiveness.

3 Proposed system overview

Figure 2 shows the whole description of the proposed autoscaling framework. The proposed autoscaling framework is composed of four major components.

i. **Workload generation**: we have used the HTTPerf, which selects the exposed URLs of the benchmark application and sends the requests on these URLs. We will see a detailed explanation in sub-section 3.1.
ii. **Performance traces collection**: recent studies have used the pre-fixed resource allocation metrics such as the number of the CPU cores for the autoscaling of the microservices. If a microservice is assigned one CPU, all its pods will be scaled in or out with one CPU. The run-time decisions of the resource allocation considering the CPU cores are neglected. Therefore, we observed the impact of varying CPU cores $\{p_{m1}, p_{m2}, p_{m3}, \cdots, p_{mn}\}$ allocated to each microservice during the autoscaling. We further explained it in sub-section 3.2
iii. **Resource prediction model training**: we pre-processed the collected traces and tested different machine learning models, i.e., linear regression, polynomial regression, lasso, ridge, SVR, decision tree, random forest, and MLP, for better prediction of the cpu cores $\{p_{m1}, p_{m2}, p_{m3}, \cdots, p_{mn}\}$ of each microservice along the number of containers $\{c_{1.k}, c_{2.k}, c_{3.k}, \cdots, c_{n.k}\}$. Further, we explained it in sub-section 3.3.
iv. **Workload forecasting**: in sub-section 3.4, we forecasted the workload based on the last $a_t, a_{t-1}, \cdots, a_k$ observations and used the forecasted workload in the prediction of the CPU cores and the corresponding number of the required pods for each microservices.
v. **Autoscaling controller**: we compared the previously used CPU cores and the number of the pods with the predicting CPU cores and the number of the pods for each microservice and dynamically scaled in or scaled out the resources when we found the differences. Moreover, We discussed the working of the controller in sub-section 3.6

3.1 Workload generation

This component uses HTTPerf as the load generator to generate load on the Google-provided benchmark application for testing microservice architecture. We have explained the benchmark application in sub-section 4.1. To mimic actual world workload, load generation frequency is a critical factor. Therefore, we used five real-world workloads - WorldCup, Wikipedia, Clark, Calgary, and Nasa - as discussed in sub-section 4.2. This section is intentionally placed to provide a system overview and introduce the workload generation and division aspect, aiming to give readers a comprehensive understanding at an early stage. In real-world applications, each microservice receives a varying incoming workload. For instance, on shopping websites, people tend to browse products more than buying, resulting in 80 to 90% of the traffic directed toward browsing services. To simulate this behavior, we have used the benchmark URL exposed distribution. We generated load on all exposed links of the benchmark application, such that microservices with more exposed links received more traffic. For example, in the case of a product catalog service with five products, we used all links to generate

Table 1 Comparison of the proposed method with the existing state-of-the-art

Method	Autoscaling strategy	Evaluation metrics	Workload forecast	Microservice	Multiple services	Platform	Container and CPU core Scaling	Technique used
Rossi et al. [33]	Predictive	CPU utilization and memory usage	No	Yes	No	Cloud	No	Reinforcement learning
Abdullah et al. [16]	Reactive and predictive	CPU utilization	Yes	Yes	No	Fog	No	Machine learning
Yu et al. [34]	Rule-Based	latency	No	Yes	Yes	Cloud	No	Bayesian optimization
Yan et al. [35]	Reactive and predictive	CPU utilization	Yes	Yes	Yes	Cloud	No	Machine learning and reinforcement learning
Hossen et al. [36]	Feedback based scaling	Response time	No	Yes	Yes	Cloud	No	Rule-based
Ruíz et al. [37]	Reactive scaling	CPU utilization and memory usage	No	Yes	No	Cloud	No	Rule-based
Razavi et al. [38]	Predictive scaling	Throughput and latency	No	No	No	Cloud	No	Deep learning
Abdullah et al. [17]	Predictive scaling	Response time and number of requests	Yes	Yes	No	GKE	No	Machine learning
Khaleq et al. [39]	Predictive	CPU utilization and response time	No	Yes	No	GKE	No	Deep learning
Sachidananda et al. [40]	Reactive	CPU utilization	No	Yes	Yes	Cloud	No	Bayesian optimization
Ezzeddine et al. [41]	Reactive and proactive	CPU utilization	Yes	Yes	No	Cloud	No	Rule-based and machine learning
Zafeiropoulos et al. [42]	Predictive	CPU utilization	No	No	No	Cloud	No	Deep learning
Toka et al. [43]	Reactive and predictive	CPU utilization	Yes	No	No	Cloud	No	Rule-based, machine learning, deep learning, and reinforcement learning
Dang-Quang et al. [44]	Proactive	HTTP request rate	Yes	No	No	Cloud	No	Deep learning
CDAScaler	Predictive	Response time	Yes	Yes	Yes	Cloud	Yes	Machine learning

more load on the product catalog service compared to the checkout service [48]. To provide insight into the workload distribution over the different microservices, we present Table 2.

3.2 Performance traces collection

In this component, we discussed performance traces exploration. We have used Kubernetes for managing the benchmark microservices application. We have represented the microservices with the $\{m_1, m_2, m_3, \cdots, m_n\}$ where m_n represents the nth microservice of the benchmark application. Each microservice is deployed with resource metrics such as CPU cores. We represented the CPU pins/cores with $\{p_{m1}, p_{m2}, p_{m3}, \cdots, p_{mn}\}$ for each microservice where p_{m1} represents that cores of microservice one and p_{mn} represents the number of cores for the nth microservie.

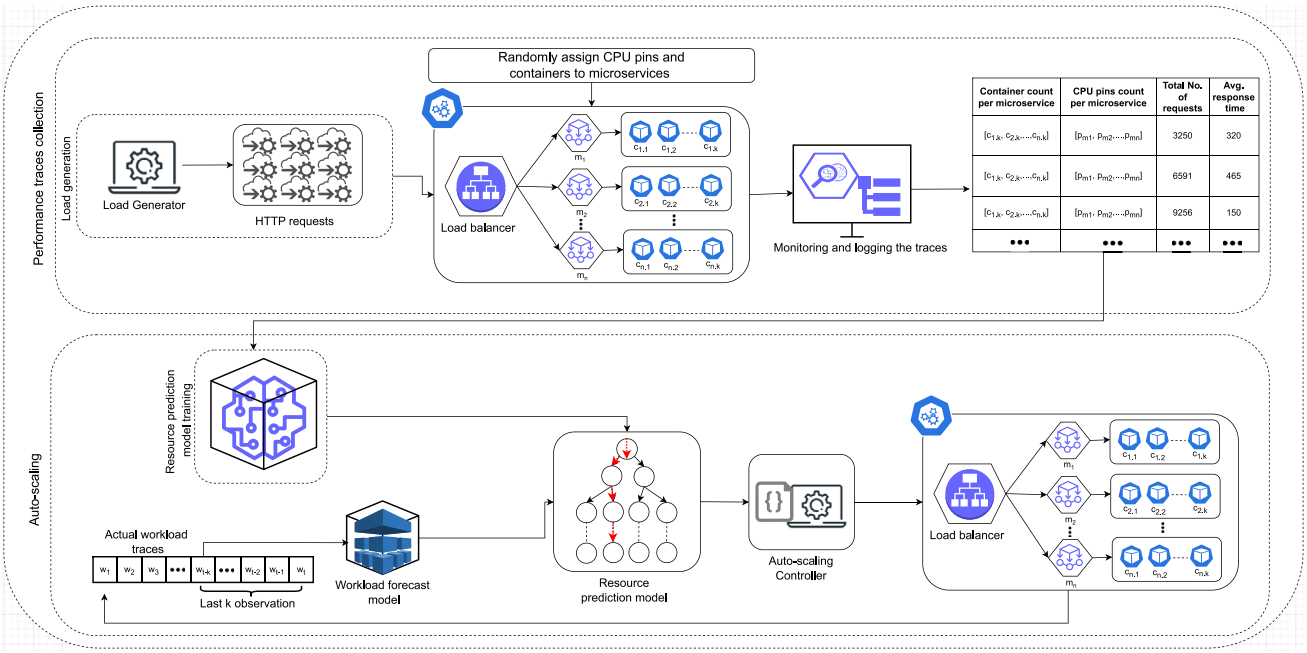


Fig. 2 CDAScaler system diagram depicting the resource provisioning model training, workload forecasting, configuration selecting, and deploying of the chosen configuration for multiple microservices

Table 2 Request distribution for different microservices of the benchmark application

Service name	Home	Product catalog	Checkout	others
Load Percentage	83%	8.5%	6%	2.5%

Each microservice can have multiple container counts based on the incoming load. We have used the $\{c_{1.k}, c_{2.k}, c_{3.k}, \cdots, c_{n.k}\}$ notation to show the running number of the containers for each microservice $c_{n.k}$ shows the container count for the microservice n. We found that more requests are passed through the frontend service, so it is bottle naked and important to keep it under observation. Secondly, we found the product catalog service more saturated because many products are available on the website. All remaining services did not cross the CPU limit and showed increased response time even in bursty workload. So, managing the CPU cores and containers count of the frontend and product catalog was challenging. That's why we discussed the frontend and product catalog service in our experiment. We tested the different numbers of CPU cores and container count combinations on both of these services and collected the traces. Then pre-processing was applied to the collected traces and removed those traces which gave more response time than the specified threshold for observing the service level agreement.

3.3 Resource prediction model training

In this step, we train our model to predict the CPU resource and number of containers for each microservices. We used linear regression, polynomial regression, lasso, ridge, decision tree, random forest, and multiplayer perceptron for training. We used 80% data for training and 20% for testing. Then we applied hyperparameter tuning to all these algorithms and calculated the mean square error. We chose the algorithm that gave minimum MSE.

3.3.1 Learning models

We have collected the application performance traces based on different deployments. Each deployment had a different number of the core and container count. In preparing the dataset, we have selected only those traces that did not violate the response time. Equation 1 shows the prediction model.

$$\mathbb{C}_{t+1}, \mathbb{P}_{t+1} \leftarrow \Phi(\widehat{a}_{t+1}, \tau_{sla}) \tag{1}$$

We evaluated the different machine learning (ML) models, which include the linear regression (LR), polynomial regression (PR), lasso, ridge, support vector regressor (SVR), decision tree (DT), random forest (RF), and multi-layer perceptron (MLP). In the prediction model, we gave the total number of requests with the 95th percentile of the response time to predict the CPU cores and containers count for the microservices. $\mathbb{C}_{t+1}$ indicates the predicted

containers sets, and $\mathbb{P}_{t+1}$ shows the expected resource(number of CPU) for the microservice. We used 80% data for the training and 20% for the testing. We calculated the mean square error(MSE) of each above-described ML method and selected a method showing a minimum MSE value. The evaluation of ML methods is discussed in Sect. 3.3.2. We do not use the deep learning model as they are used for complex and higher dimensional data to learn important features and hierarchical representations. However, for our specific problem, which involves a lower-dimensional dataset and a simpler learning requirement, employing traditional machine learning models is easy to train with good accuracy.

3.3.2 Model evaluation

The prediction model predicts the number of cores and container count for the microservices. Table 3 shows the mean square error of the different machine learning algorithms during the training. Linear and polynomial regression gave more MSE errors and performed the most sinister compared to the other learning algorithms. The decision tree gave minimum MSE and transcended all tested machine learning algorithms. We choose the decision tree algorithms for the prediction.

3.3.3 Decision tree

The decision tree is a widely used prediction model. It is summoned as CART (classification and decision tree) and ID3. The decision tree is upside down, in which each node represents some conditions, and each edge represents the decision. The last nodes are leaf nodes representing the class labels or predicted values. Decision trees usually are of two types. Categorical decision trees are used for classification, and continuous decision trees are used for continuous value prediction.

We have used the CART decision tree for our experiment. Because in ID3 algorithms, features get more important whose count number is more. In CART, node n, the tree is represented as t. The tree exists in the form of a binary tree and each node is split into two subtrees $t_l(\vartheta)$ and $t_r(\vartheta)$. The cost function of the decision tree is given in Eq. 2.

$$\Psi(T,\vartheta) = \frac{M_l}{M} I(t_l(\vartheta)) + \frac{M_r}{M} I(t_r(\vartheta)) \tag{2}$$

where ϑ represents the $\vartheta = (x, t_x^n)$. Here x represents the input features, and t_x^n represents the threshold for the feature. H represents the impurity calculated in Eq. 3.

$$I(X_n) = \frac{1}{N}\sum_{i=1}^{N}(n_i - \mu_N)^2, \tag{3}$$

where $I(X_n)$ represents the training data at node n. $\frac{1}{N}\sum_{i=1}^{N}$ show prediction based on the training. We select the feature which gave the minimum impurity as shown in Eq. 4.

$$\vartheta^* = \text{argmin}_{\vartheta}\Psi(t,\vartheta) \tag{4}$$

3.4 Workload forecasting

Our proposed solution uses the forecasted workload to predict the CPU cores and containers count of each microservice for the next time interval a_{t+1}. Forecasted workload helped to allocate resources before time to avoid SLA violations. For training, the same number of example points are fed to the model when training example points are less than k. But when the count is more than k, the last k training points are used for workload forecasting.

3.4.1 Workload forecasting model

We used an online workload forecasting approach for timely resource provisioning and to avoid SLA violations. We have used the linear regression model based on the last k interval workload log for forecasting [17]. Linear regression outperforms for small intervals datasets [49]. Advanced machine learning models like LSTM are effective for capturing long-term dependencies and patterns in data but are computationally intensive, slow to train, and complex to implement. Moreover, deep learning methods are commonly used for complex and higher dimensional data to learn important features. Moreover, time series forecasting models, such as ARIMA, are unsuitable for predicting online cloud workloads using short-duration data. This is mainly due to the absence of seasonality and trends in the short-duration workloads. This is a well-established fact, mentioned in a notable work [50]. Hence, we employ Linear Regression, functioning as a piecewise model, to train individual linear models within each

Table 3 MSE of different machine learning models to learn the run-time configuration of the microservice

Methods	LR	PR	Lesso	Ridge	SVR	DT	RF	MLP
MSE	15.34	13.21	12.91	12.94	13.41	**10.06**	11.44	12.46

workload interval. This approach allows us to approximate local trends within the current intervals of the workload data. The fundamental concept behind piecewise linear regression is to adapt the regression function in "pieces" when the data exhibit distinct linear trends across various intervals. This piecewise linear regression strategy yields good results in capturing local trends in short-duration cloud workloads.

Liner regression used the last k observation of the workload at a given time interval t and predicted the $\tilde{a}_{t+1}$. We used $\tilde{a}_{t+1} = \vartheta_1 a_t + \vartheta_0$ for the forecasting. It is also shown in the following equation.

$$X\begin{pmatrix}\vartheta_0\\ \vartheta_1\end{pmatrix} = Y$$
$$X = \begin{pmatrix}1 & a_{t-1}\\ 1 & a_{t-2}\\ \vdots & \vdots\\ 1 & a_{t-k}\end{pmatrix} \quad Y = \begin{pmatrix}a_t\\ a_{t-1}\\ \vdots\\ a_{t-k+1}\end{pmatrix} \tag{5}$$

Where X shows the last k intervals of the workload and Y shows the predicted workload for the time interval a_{t+1}. ϑ_0 and ϑ_1 are the learnable parameters we will learn by minimizing the cost function. The following equation shows the state-of-the-art cost function.

$$L(\hat{\vartheta_1}) = \sum_{i=1}^{n}\left(y_i - a_i'\hat{\vartheta_1}\right)^2 \tag{6}$$

After minimizing the loss function, the following equation shows the forecasted workload after learning the ϑ_0 and ϑ_1

$$\widehat{a}_{t+1} = \begin{pmatrix}1\\ a_t\end{pmatrix}^{\top}\begin{pmatrix}\vartheta_0\\ \vartheta_1\end{pmatrix}. \tag{7}$$

3.5 Multi-objective optimized autoscaling controller

Autoscaling controller works on multi-objective optimization problems. In this model, We first formulate resource allocation as a multi-objective optimization problem. The objective is to minimize the overall resource allocation while considering adjustments in container count ($\mathbb{C}_{t+1}$) and core allocation ($\mathbb{P}_{t+1}$):

$$\text{Minimize} \sum_{i=1}^{N}(\mathbb{C}^i_{t+1} + \alpha \cdot \mathbb{P}^i_{t+1}) \tag{8}$$

where N is the number of microservices. $\mathbb{C}^i_{t+1}$ is the predicted container count for microservice i. $\mathbb{P}^i_{t+1}$ is the predicted core allocation for microservice i. α is a weighting factor representing the importance of core allocation relative to container count. Decision Variables $\mathbb{C}^i_{t+1}$ shows the Predicted container count for microservice i and $\mathbb{P}^i_{t+1}$ shows the Predicted core allocation for microservice i.

Our optimization model consists of three key constraints: Workload Forecasting, Resource Adjustment, and Integer Constraints. The objective is to ensure that the predicted workload adheres to the workload forecasting model, satisfying the specified SLA threshold.

$$\widehat{a}^i_{t+1} \leq \tau_{sla} \quad \forall i \in [1, N] \tag{9}$$

Resource Adjustment: The differences in container count and core allocation should be consistent with the predicted values:

$$\begin{aligned}\Delta^i_c &= \mathbb{C}^i_{t+1} - \mathbb{C}^i_t \quad \forall i \in [1, N]\\ \Delta^i_p &= \mathbb{P}^i_{t+1} - \mathbb{P}^i_t \quad \forall i \in [1, N]\end{aligned} \tag{10}$$

Integer Constraints: Container count and core allocation should be non-negative integers:

$$\mathbb{C}^i_{t+1}, \mathbb{P}^i_{t+1} \geq 0 \quad \forall i \in [1, N] \tag{11}$$

This optimization model encapsulates the essence of the autoscaling approach, aiming to find the optimal predicted container count and core allocation for each microservice while minimizing the overall resource allocation cost and ensuring workload forecasting adherence and SLA compliance. The weighting factor α allows for adjusting the importance of core allocation relative to container count in the objective function.

3.6 Algorithm for autoscaling controller

The autoscaling controller controls the decision of the resource provisioning for each microservices. Algorithm 1 shows the working of the proposed system, including the autoscaling controller. For the successful working of the proposed system, it needs the application monitoring time interval (ξ), SLA threshold (τ_{sla}), window size for workload forecasting (k), workload forecasting model, which is $\psi(a_t, a_{t-1}, \cdots, a_k) \rightarrow \widehat{a}_{t+1}$, currently allocated set of containers $\mathbb{C}_t$ to the microservices, allocated set of cores $\mathbb{P}_t$ to predict the output of the container set $\mathbb{C}_{t+1}$ and cores $\mathbb{P}_{t+1}$.

Algorithm 1 Algorithm for the CDA-Scaler.

Input: Application monitoring interval (ξ), Response time SLA threshold (τ_{sla}), Window size for workload forecasting (k), Workload forecasting model $\psi(a_t, a_{t-1}, \cdots, a_k) \rightarrow \widehat{a}_{t+1}$, Running set of containers $\mathbb{C}_t$ to each microservices, Running set of cores $\mathbb{P}_t$ to each microservices.

Output: Predicted container set $\mathbb{C}_{t+1}$ and cores $\mathbb{P}_{t+1}$

while *true* **do**
  Wait for ξ seconds
  $\widehat{a}_{t+1} \leftarrow \psi(a_t, a_{t-1}, \cdots, a_k)$
  $\mathbb{C}_{t+1}, \mathbb{P}_{t+1} \leftarrow \Phi(\widehat{a}_{t+1}, \tau_{sla})$ ▷ `Predict the containers count and cores for each microservice.`
  if $|\mathbb{C}| == \mathbb{C}_{t+1}$ *and* $|\mathbb{P}| \neq \mathbb{P}_{t+1}$ **then**
    $\Delta_p \leftarrow \mathbb{P}_{t+1} - \mathbb{P}_t$ ▷ `Calculate the difference between predicted and actual cores`
    if $\Delta_p > 0$ **then**
      $\mathbb{P} + \Delta_p$ ▷ `Add the` Δ_p
    else if $\Delta < 0$ **then**
      $\mathbb{P} - \Delta_p$ ▷ `Delete the` Δ_p
    end
  else if $|\mathbb{C}| \neq \mathbb{C}_{t+1}$ *and* $|\mathbb{P}| == \mathbb{P}_{t+1}$ **then**
    $\Delta_c \leftarrow \mathbb{C}_{t+1} - \mathbb{C}_t$ ▷ `Calculate the difference between predicted and actual container count`
    if $\Delta_c > 0$ **then**
      $\mathbb{C} + \Delta_c$ ▷ `Add the` Δ_c
    else if $\Delta_c < 0$ **then**
      $\mathbb{C} - \Delta_c$ ▷ `Delete the` Δ_c
    end
  else if $|\mathbb{C}| \neq \mathbb{C}_{t+1}$ *and* $|\mathbb{P}| \neq \mathbb{P}_{t+1}$ **then**
    $\Delta_c \leftarrow \mathbb{C}_{t+1} - \mathbb{C}_t$ ▷ `Calculate the difference between predicted and actual container set`
    if $\Delta_c > 0$ **then**
      $\mathbb{C} + \Delta_c$ ▷ `Add the` Δ_c
    else if $\Delta_c < 0$ **then**
      $\mathbb{C} - \Delta_c$ ▷ `Delete the` Δ_c
    end
    $\Delta_p \leftarrow \mathbb{P}_{t+1} - \mathbb{P}_t$ ▷ `Calculate the difference between predicted and actual cores`
    if $\Delta_p > 0$ **then**
      $\mathbb{P} + \Delta_p$ ▷ `Add the` Δ_p
    else if $\Delta_p < 0$ **then**
      $\mathbb{P} - \Delta_p$ ▷ `Delete the` Δ_p
    end
  end
  $t \leftarrow t + 1$
end

First of all, the proposed system forecast the workload $\widehat{a}_{t+1}$ based on the last $(a_t, a_{t-1}, \cdots, a_k)$ intervals and feed it to the machine learning model which predicts the container count $\mathbb{C}_{t+1}$ and cores $\mathbb{P}_{t+1}$ for each microservice. Further, the Controller received the predicted set of the cores $\mathbb{P}_{t+1}$ and container count $\mathbb{C}_{t+1}$ and compared the existing set of container count $\mathbb{C}_t$ and cores $\mathbb{P}_t$ for each microservices.

The Controller updates the changes in three different scenarios. When the container's count is equal, and the cores differ, the Controller calculates the difference between predicted and running cores Δ_p. The Controller adds the resource if the difference is more significant than zero. If the difference is less than zero, the Controller decreases the core resources according to the calculated difference. Second, when cores are equal and the container

count is different, the Controller calculates the Δ_c based on the running, predicts the container count, and increases and decreases the container count based on the calculated Δ_c value. Third, when both cores and container count are different, the Controller calculates the difference of both cores and container count and updates the container count and cores based on the calculated Δ_p and Δ_c. After updating the changes, the system waits for the time interval (ξ) and repeats the same steps.

4 Experimental design

This section describes the testbed infrastructure, benchmark application, workloads for experiments, evaluation matrix, and baseline experiments. We evaluate the proposed strategy with five real-time workload traces. We also implement the most widely used strategies to compare our proposed solution's performance, as explained in the following sub-sections.

4.1 Benchmark application

We used a 10-tier cloud-native microservices benchmark [18]. This benchmark is a web-based e-commerce website known as an online boutique application. In this application, a user can explore the e-commerce items, add them to the cart, and purchase any number of items. Services communicate through gRPC (google remote procedure call) and are written in different languages, such as Python, JavaScript, Node-JS, Go, and C#. Figure 3 shows the architecture diagram of the online boutique application. . We have highlighted the fronted and product catalog service with the red indicating the more saturated microservice. It is very similar behavior to a real-world shopping application. Normally users search the products and move around the products than buying them products. Keeping this in mind, in the results section, we discussed the maximum number of containers for these two services.

4.2 Workloads for experiments

We used five real-time workloads for the experiment, which included the web traces of world-Cup, Wikipedia, ClarkNet, Calgary, and NASA. Figure 4 shows these five real workloads. These workloads are publicly available for experimental purposes.

The traces for WorldCup were collected during the FIFA WorldCup of 1998, and these are publicly available for real-time simulations [51]. The Wikipedia web traces were obtained from more frequently used wikis. It observed a typical pattern in which traffic comes in a rusty way [52]. The Clarknet is a real-time workload collected over two weeks. It shows the cyclic behavior because traffic was heavy in the daytime compared to night and weekends [53]. Calgary contained the one-year HTTP requests collected at Calgary's Department of Computer Science [54]. The Nasa workload was collected at Space Center in Florida. It shows a repeated pattern over the period, which makes it more realistic. It also has small spikes in workload [55].

4.3 Testbed infrastructure

We used the Kubernetes cluster as a testbed to conduct the proposed solution. We used three machines with the specification of Core-i7 with 8-GB RAM and 2-TB hard disk. One node acts as a controller node, and the other two are agent nodes. The controller node is responsible for scheduling multiple microservices on agent nodes.

4.4 Evaluation metrics

As an evaluation matrix, we utilized various performance metrics, including the total number of processed requests, response time, cost, and scale operations. These performance traces were gathered from different baseline methods, namely reactive autoscaling based on CPU utilization (C-React), proactive autoscaling based on CPU utilization (C-Proc), proactive autoscaling based on the resource instance count (RC-Proc), static over-provisioning (static_op), static under-provisioning (static_up), and CDAScaler. Subsequently, we compared these methods' performance to show the proposed solution's superiority.

1. The number of processed requests is one key parameter for testing the application performance. The proposed solution gave more processed requests than the baseline and static_up and was approximately equal to the static_op. Static_op gave more processed requests because we have allocated extra resources with higher cost, which is not cloud user-friendly.
2. Response time to process the request is a prominent feature in cloud computing. We collected the response time of all the experiments and compared them. The proposed solution's response time is relatively less than the baseline and static_up.
3. Every cloud service provider tries to provide optimal resources with minimal cost to attract users. To show the effectiveness of our proposed method, we have also calculated the total cost of each experiment. We assume the 1.0$ per minute cost for each running container, similar to Amazon Web Services (AWS) service model. The proposed method gave a cost-effective solution as compared to the other baseline experiments.

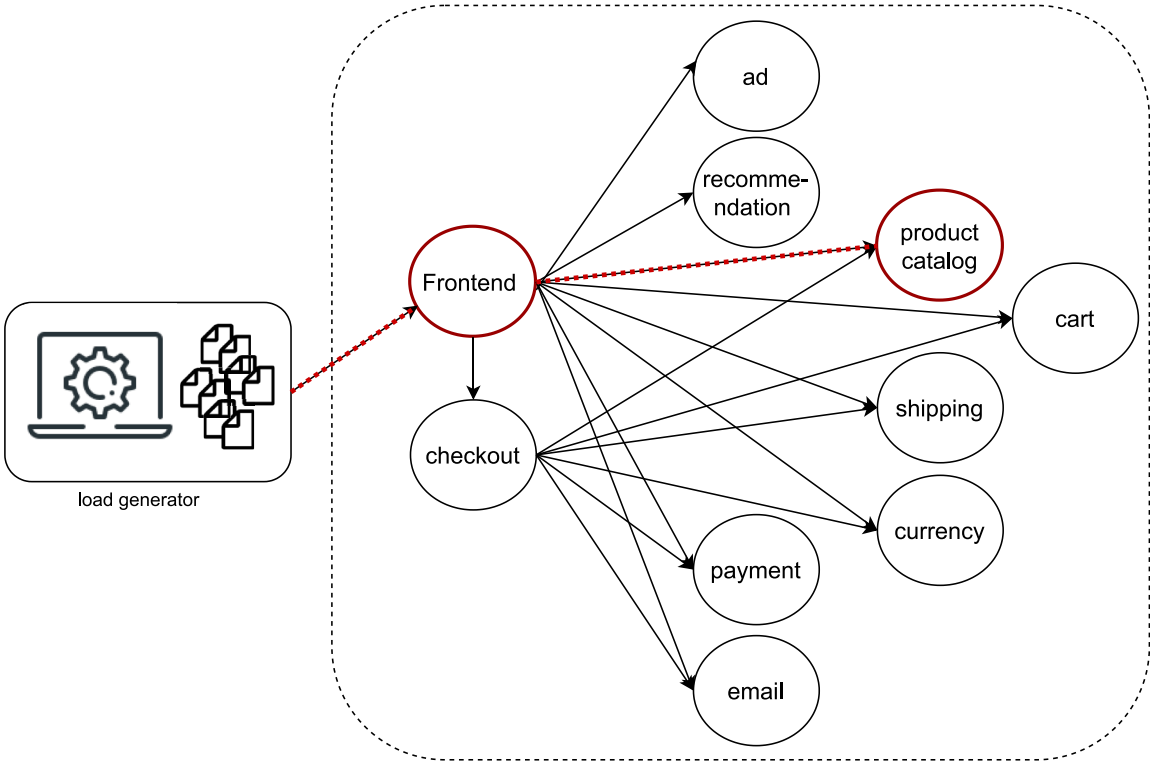


Fig. 3 Benchmark of multiple microservices pointing up the more resource-demanding services

4. Scale operations show the oscillation behavior of the application based on the underlying resource allocation strategy. The proposed solution gave fewer scale operations than the other, showing less oscillating behavior.

4.5 Baseline experiment

Many strategies have been proposed to increase the performance of the applications in terms of response time and latency. In these strategies, resources are provided automatically based on reaction and prediction. In reactive scaling, resources are provided based on the reaction, which means that when a service crosses the CPU limit from a specified threshold, then more resources are added to keep CPU utilization below the threshold and meet the SLA. In contrast, predictive strategies predict the next event based on historical data. Primarily, prediction is performed on incoming workload, response time, latency, and CPU. Machine learning, deep learning, and reinforcement learning are frequently used for prediction. In this article, we have compared our proposed strategy with three different baseline methods, including *Autoscaling Based on CPU Utilization (C-React)* [56], Proactive *Autoscaling Based on CPU Utilization (C-Proc)* [16], and *Proactive Autoscaling Based on Resource Instance Count (RC-Proc)* [17]. The C-Proc and RC-Proc are our previous state-of-the-art works, which outperform the works presented by other researchers [57–61]. Therefore, we compared the proposed work with our previous works, which already showed superiority to other existing methods. These three baseline strategies are briefly discussed in the following sub-sections. These three baseline strategies are briefly discussed in the following sub-sections.

4.5.1 Reactive autoscaling based on CPU utilization (C-React)

We have implemented reactive autoscaling inspired by [56]. The authors used the CPU threshold to scale in and scale out the resources. When a particular service crosses the CPU utilization above 70%, then the new pods of the respective service are launched, and the Kubernetes cluster divides the workload equally on the running pods of that particular service. In contrast, when one specific service CPU goes down to 30%, the rule-based strategy checks the running number of the pods and scales out the service by one when it has more than one number of the pods.

4.5.2 Proactive autoscaling based on CPU utilization (C-Proc)

We reproduced the results of predictive autoscaling proposed by [16]. The authors used both reactive and proactive

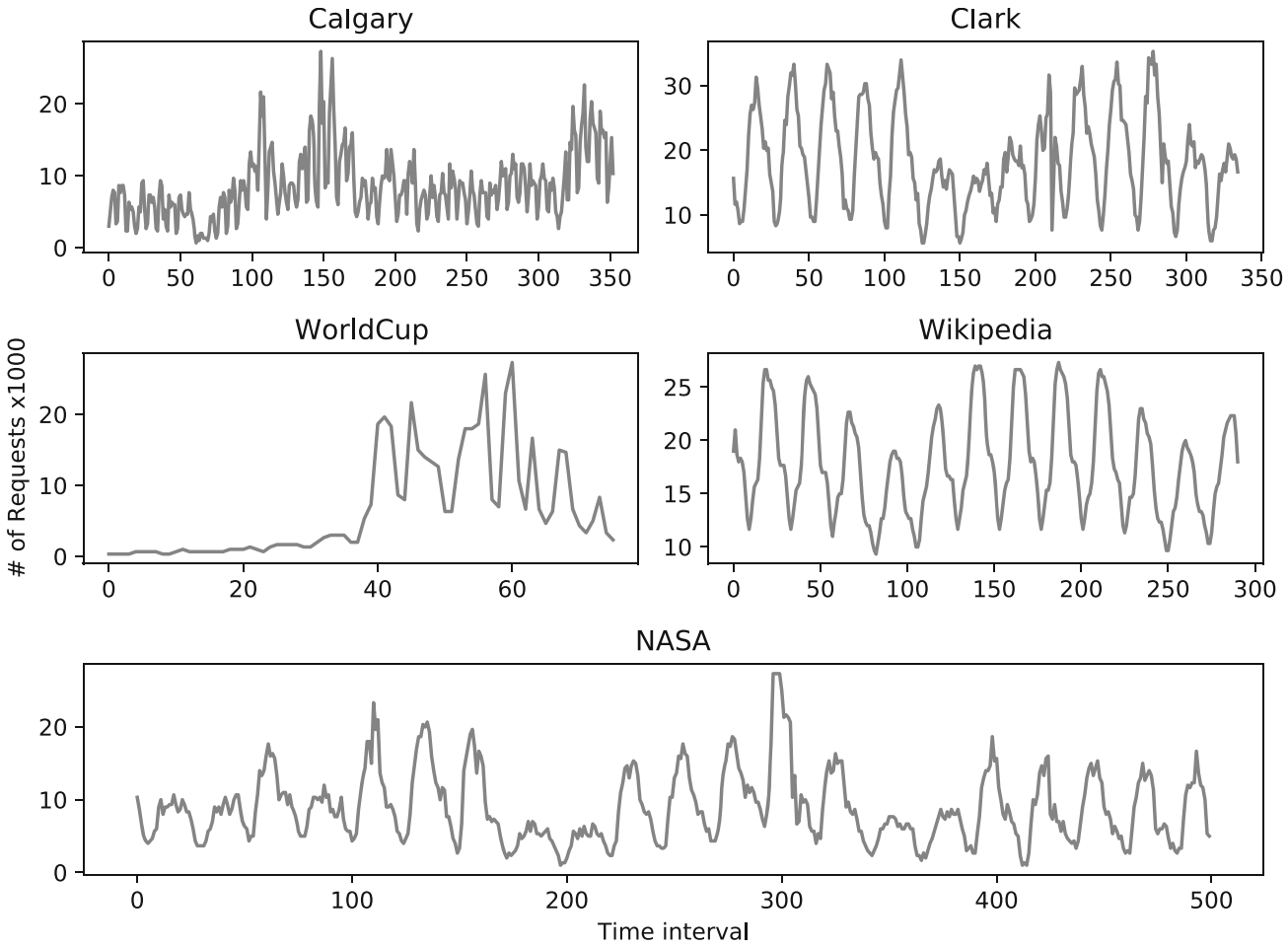


Fig. 4 Real world workloads

strategies. In this strategy, authors employed CPU utilization as an evaluation matrix. In the first step, the reactive approach is used so that a model can learn scaling policy, and after getting training, the model shifts from reactive scaling to predictive scaling. In predictive scaling, the model decides on the last k window size and predicts the coming workload. When the forecasted workload violates the threshold of the upper bound, more resources are added to lower the CPU utilization. While in contrast, when the forecasted CPU of a particular service goes down to a specific threshold, then strategy checks the number of containers and minus it by one.

4.5.3 Proactive autoscaling based on resource instance count (RC-Proc)

We have implemented the proposed solution by Abdullah M et al. [17]. This algorithm dynamically scales the microservices based on forecasted workload, response time, and the number of containers. This algorithm is divided into two main components. The first component model is trained offline on the number of requests, response time, and the number of containers, and the second component uses the regression model, which predicts the total number of requests in the next time interval based on the previous k window size. Then forecasted workload is fed into the first component to predict the number of containers with minimum response time.

5 Experiments results and evaluations

In this section, we present and discuss the experimental results during the evaluations of the proposed system and compare them with the different baseline methods, including C-React, C-Proc, and RC-Proc. To show the effectiveness of our proposed method, we also compare it with the static under-provisioning (Static_up) and over-provisioning (Static_op) allocation.

Table 4 shows the percentage of total process requests, unprocessed requests percentage, SLA violations percentage, cost, number of maximum containers for different

microservices with the maximum number of CPU cores, and the count of scale operations for the proposed and baseline autoscaling methods using five different real-time workloads, including WorldCup, Wikipedia, ClarkNet, Calgary, and NASA.

An autoscaling method yields the maximum percentage of processed requests, and the minimum number of unprocessed requests and SLA violations is considered best. Moreover, an autoscaling method with minimum cost and less scale operation count is also considered best. The proposed method excels in all evaluation metrics as compared to the baseline methods and is closer to the static over-provisioning method with very less cost. Our proposed method yields 1.03× to 1.06×, 1.00× to 1.04×, and 1.00× to 1.04× more processed requests for various workloads as compared to C-React, C-Proc, and RC-Proc baseline methods respectively. The proposed solution reduces unprocessed requests by 3.11× to 97.77×, 2.13× to 94.76×, and 1.53× to 40.82× by using C-React, C-Proc, and RC-Proc baseline methods respectively for different workloads. Moreover, the proposed method yields fewer SLA violations by 15.28× to 78.23× against C-React, 13.53× to 72.96× against C-Proc, and 1.36× to 4.62× against RC-Proc across different workloads. In terms of cost, the proposed strategy incurs 1.09× to 8.18× lower costs compared to C-React, 1.09× to 4.27× lower costs compared to C-Proc, and 1.15× to 7.55× lower costs compared to RC-Proc across all workloads.

Figure 5 compares the proposed method with baseline methods (C-React, C-Proc, and RC-Proc). The figure shows the average percentage of processed requests, unprocessed requests, SLA violations, cost, and scale operations under the different workloads. The processed requests bar graph has 0.67%, 2.60%, and 3.52% increase in processed requests for the C-Proc, RC-Proc, and proposed solution compared to the C-React baseline method, which shows that the proposed solution outperforms as compared to the baseline methods. The unprocessed requests sub-graph depicts the 0.67%, 2.6%, and 3.5% decrease in requests over the C-React baseline, showing the proposed solution's superiority. The SLA violation sub-graph shows the 5.4%, 39.8%, and 41.04% decrease in SLA violation. Almost a 41% decrease in the violations shows that the proposed solution outperforms more than the others. The cost sub-graph shows an average of 302$, 436$ more for C-Proc and RC-Proc, and 97$ less cost of proposed as compared to the C-React baseline method. An

Table 4 Comparison of the proposed solution with C-React, C-Proc, and RC-Proc over the five real-world workloads

Workload	Autoscaling Method	Processed Requests (%)	Unprocessed Requests (%)	SLA Violations (%)	Cost ($)	Max. Containers Count		Max. cores Count		Scale Operation's Count
						Frontend	Product	Frontend	Product	
WorldCup	C-React	96.54	3.46	41.67	**211**	4	2	1	1	32
	C-Proc	96.74	3.26	39.95	230	4	2	1	1	19
	RC-Proc	99.57	0.43	0.95	313	3	3	1	1	16
	CDAScaler	**99.91**	**0.09**	**0.70**	212	**4**	**2**	**2**	**1**	**12**
Wikipedia	C-React	93.66	6.34	60.43	1323	4	2	1	1	98
	C-Proc	95.90	4.10	42.57	1572	5	3	1	1	63
	RC-Proc	97.63	2.37	3.37	1597	4	3	1	1	60
	CDAScaler	**99.16**	**0.84**	**1.44**	**971**	**5**	**4**	**3**	**2**	**52**
Clarknet	C-React	95.82	4.18	44.89	1473	4	2	1	1	140
	C-Proc	95.91	4.09	49.87	1795	5	4	1	1	85
	RC-Proc	98.33	1.67	4.03	1794	4	3	1	1	106
	CDAScaler	**98.91**	**1.09**	**2.94**	**1163**	**6**	**3**	**4**	**2**	**78**
Calgary	C-React	97.89	2.11	24.46	1111	3	2	1	1	180
	C-Proc	98.56	1.44	14.80	1590	5	3	1	1	94
	RC-Proc	98.93	1.07	2.13	1696	4	3	1	1	166
	CDAScaler	**99.32**	**0.68**	**1.09**	**1101**	**5**	**4**	**3**	**1**	**22**
NASA	C-React	95.75	4.25	40.45	**1423**	3	2	1	1	159
	C-Proc	95.88	4.12	37.73	1867	4	2	1	1	103
	RC-Proc	98.22	1.78	2.39	2325	3	3	1	1	186
	CDAScaler	**99.96**	**0.04**	**0.52**	1608	**4**	**2**	**2**	**1**	**54**

Bold values show the result of the proposed system (CDAsclaer)

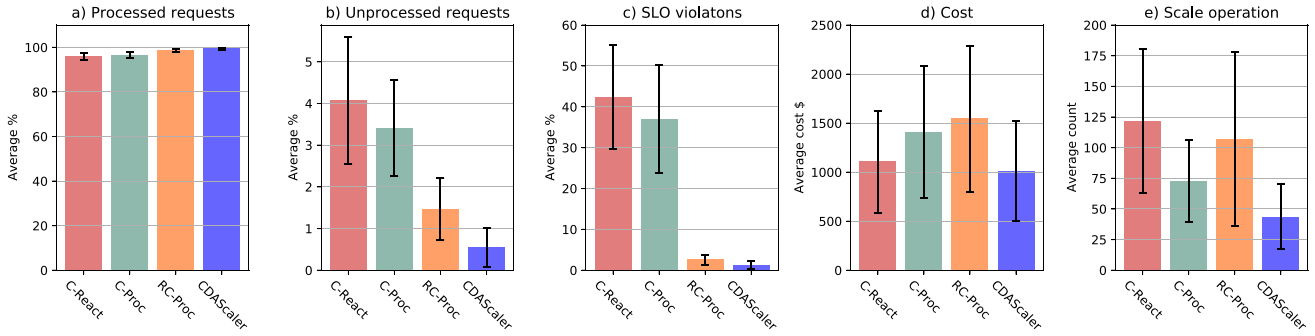


Fig. 5 Processed requests, unprocessed requests, SLA violations, cost, and scale operations comparison of the proposed solution (CDAScaler) method with baseline methods (C-React, C-Proc, and RC-Proc) using an average of the percentage of all workloads

increase in price shows that C-Proc and RC-Proc are more costly compared to the C-React baseline, while the proposed solution shows less cost with all other baselines. In the scale operation sub-graph, average 49, 15, and 78 scale operations are observed, which shows that the proposed solution is less oscillating than the others.

Figure 6 shows the processed requests, unprocessed requests, SLA violations, and response times for C-React, C-Proc, RC-Proc, and the proposed methods using the ClarkNet workload. C-React baseline method yields high SLA violations due to reactive resource allocation. Meanwhile, RC-Proc performs better, maintaining low violations and response times under varying workload. Overall, the proposed method outperforms by yielding high processing requests, minimizing SLA violations, reducing costs, and scaling operations compared to C-React, C-Proc, and RC-Proc baselines.

Figure 7 shows the response time comparison between proposed and baseline experiments. Response time of C-React and C-Proc is observed up to 2300ms, which is very high and not according to the service level agreement, causing more SLA violations. The response time of the RC-Proc remains less than 1000ms, although less but not below the SLA threshold. But when we observed the result of the proposed response time remained below 200ms in all workloads, which shows that the proposed solution outperforms as compared to the other baseline experiments.

To show the effectiveness of our proposed solution, we also compare the experimental results of the proposed method with the static under-provisioning (static_up) and over-provisioning (static_op) strategies. Table 5 shows the experimental result of the proposed CDAScaler, static_up, and static_op methods. During the static over-provisioning, extra resources are allocated to each microservices to meet the SLA. Providing extra resources incur more cost. In contrast, the proposed solution gave almost the same performance with a dynamic resource allocation strategy, only adding new resources when needed. Static_op processed 1.0003×, 0.9947×, 0.9987×, 0.9947×, and 1.0006× more requests than the proposed solution, but at the same time, it is producing 2.8679×, 2.3975×, 2.3044×, 2.5649×, and 2.4876× more cost than the proposed over the WorldCup, Wikipedia, ClarkNet, Calgary, and NASA workloads. Processed requests of static_op are not significantly large, but the proposed solution performed almost equally with almost 2.3× less cost.

In static under-provisioning, fewer resources are provided to each microservices. With fewer resources allocated to each microservice, the system struggles to handle the incoming load efficiently, resulting in slower response times and increased latency for end-users. Static_up gave 62.4893×, 18.4446×, 13.0394×, 4.0811×, and 62.3281× more unprocessed requests along the 51.8418×, 41.4434×, 12.9432×, 30.1406×, and 38.0496× more SLA violations than the proposed CDAScaler over the WorldCup, Wikipedia, ClarkNet, Calgary, and Nasa workloads respectively. Static under-provisioning causes resource contention and bottleneck. When multiple microservices compete for limited resources, it leads to performance degradation and even system failures.

Figure 8 compares the proposed method with static_op, and static_up. The figure shows the average percentage of processed requests, unprocessed requests, SLA violations, cost, and scale operations under the different workloads. Static_op outperforms in processed requests, unprocessed requests, and SLA violations over proposed and static_up. But it produces more cost, which is not acceptable to the client. But at the same time proposed giving almost the same processed requests, unprocessed requests, and SLA violations at much less cost, which is 1.46x less than static_op. Static_op shows the average of 99.68% processed requests, 0.31% unprocessed requests, 0.74% SLA violations, and an average of 2488$ cost. In contrast, the proposed gives 99.45% processed requests, 0.54% unprocessed requests, 1.33% SLA violations, and an average of 1011$ cost. The performance of static_op in terms of processed requests, unprocessed requests, and SLA violations is not more than 1% to proposed. But it is more costly compared to the proposed.

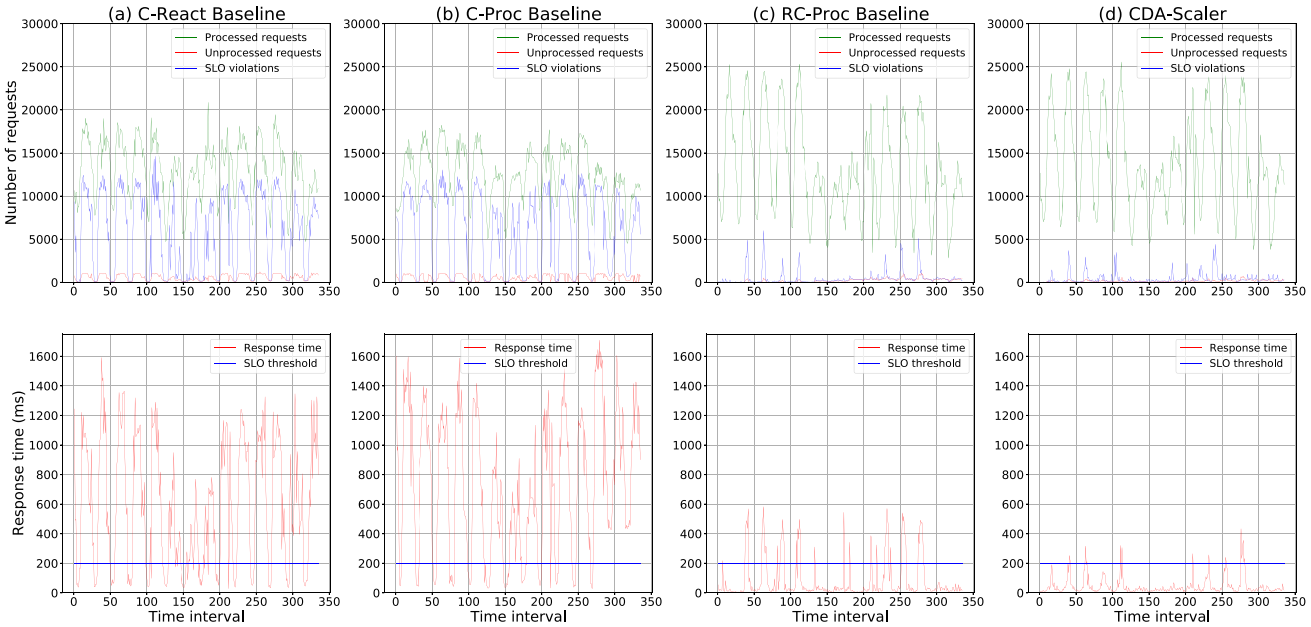


Fig. 6 Experimental results showing the processed requests, unprocessed requests, SLA violations, and response time of reactive, proactive, predictive, and proposed strategy over the ClarkNet workload

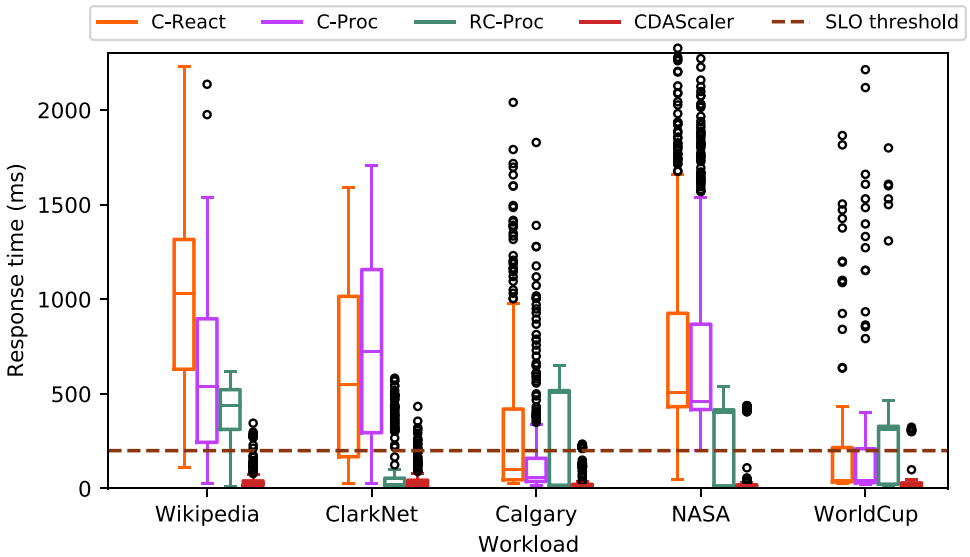


Fig. 7 Response time comparison between proposed, C-React, C-Proc, and RC-Proc baseline methods

Figure 9 shows the response time comparison between the proposed, over-provisioning, and under-provisioning experiments. Under provisioning, response time is up to 1000ms and causes more SLA violations. At the same time, the proposed and over-provisioning response time is almost identical and shows promising results. But over-provisioning is not cost-effective and causes the wastage of resources. So, we can achieve the same results with a proposed solution at less cost.

Figure 10 shows the cost comparison of C-React, C-Proc, RC-Proc, static_op, and the proposed solution with respect to static_up. For the sake of comparison, we computed the average cost percentage with respect to static_up. Static_op produced 4.3x, 4.3x, 4.3x, and 4.3x more cost relative to the static_up for Wikipedia, Clarknet, Calgary, NASA, and WorldCup. In comparison, the proposed solution produced 1.2x, 1.3x, 1.1x, 1.1x, and 0.9x more cost relative to the Static_up for Wikipedia, Clarknet, Calgary, NASA, and WorldCup. Results show that static_op is three times more costly than the proposed solution. This almost three times less cost shows that the proposed solution is more cost-effective than the static_op.

Table 5 Comparison of the proposed solution with static_op and static_up over the five real-world workloads; WorldCup, Wikipedia, ClarkNet, Calgary, and Nasa

Workload	Autoscaling method	Processed requests (%)	Unprocessed requests (%)	SLA violations (%)	Cost ($)
WorldCup	Static_op	99.95	0.05	0.41	608
	Static_up	94.56	5.44	37.07	114
	CDAScaler	**99.91**	**0.09**	**0.70**	**212**
Wikipedia	Static_op	99.69	0.31	1.09	2328
	Static_up	84.51	15.49	40.41	436.5
	CDAScaler	**99.16**	**0.84**	**1.44**	**971**
ClarkNet	Static_op	99.04	0.96	1.23	2680
	Static_up	85.76	14.24	38.13	502.5
	CDAScaler	**98.91**	**1.09**	**2.94**	**1163**
Calgary	Static_op	99.86	0.14	0.49	2824
	Static_up	97.23	2.77	15.27	529.5
	CDAScaler	**99.32**	**0.68**	**1.09**	**1101**
NASA	Static_op	99.90	0.10	0.47	4000
	Static_up	97.29	2.71	20.73	750
	CDAScaler	**99.96**	**0.04**	**0.52**	**1608**

Bold values show the result of the proposed system (CDAsclaer)

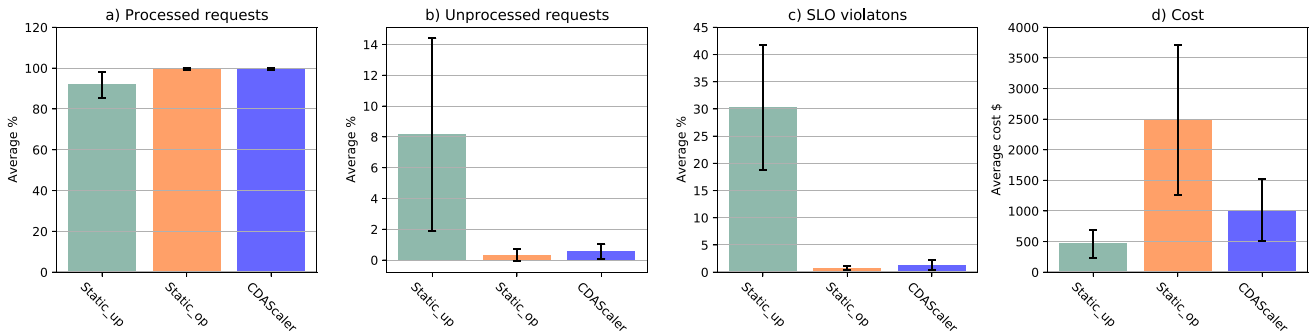


Fig. 8 Processed requests, unprocessed requests, SLA violations, cost, and scale operations comparison of the proposed solution (CDAScaler) method with static_op, and static_up using an average of the percentage of all workloads

5.1 Overview of experimental results

In our evaluation, we rigorously compare the proposed CDAScaler with existing baseline autoscaling methods, namely C-React, C-Proc, and RC-Proc. Also, we evaluated its performance against static under-provisioning (Static_up) and static over-provisioning (Static_op). Our objective is to measure its efficiency in terms of processed requests, unprocessed requests, SLA violations, and cost-effectiveness. Across diverse workloads, CDAScaler consistently demonstrated superior performance, outperforming the baseline methods by 1.01× to 1.06× in processed requests and reducing unprocessed requests and SLA violations by 7.55× to 97.77×. Moreover, it showcased cost efficiency, incurring 1.09× to 8.18× less cost compared to C-React, indicating its ability to deliver optimal results at a reduced operational cost. Additionally, CDAScaler exhibited fewer scale operations, signifying stable resource allocation and reduced oscillations, enhancing its practical applicability.

Compared with static resource allocation strategies, CDAScaler demonstrated comparable performance to static_op regarding processed requests and SLA violations. Notably, it showcased remarkable cost efficiency, maintaining a balance between performance and cost, making it an appealing choice for resource allocation in dynamic cloud environments. Furthermore, CDAScaler consistently maintained response times below the SLA threshold, emphasizing its ability to adhere to performance benchmarks. This comprehensive evaluation validates the efficiency and robustness of CDAScaler, positioning it as a promising solution for dynamic autoscaling with the potential to optimize resource allocation in real-world cloud-based applications significantly.

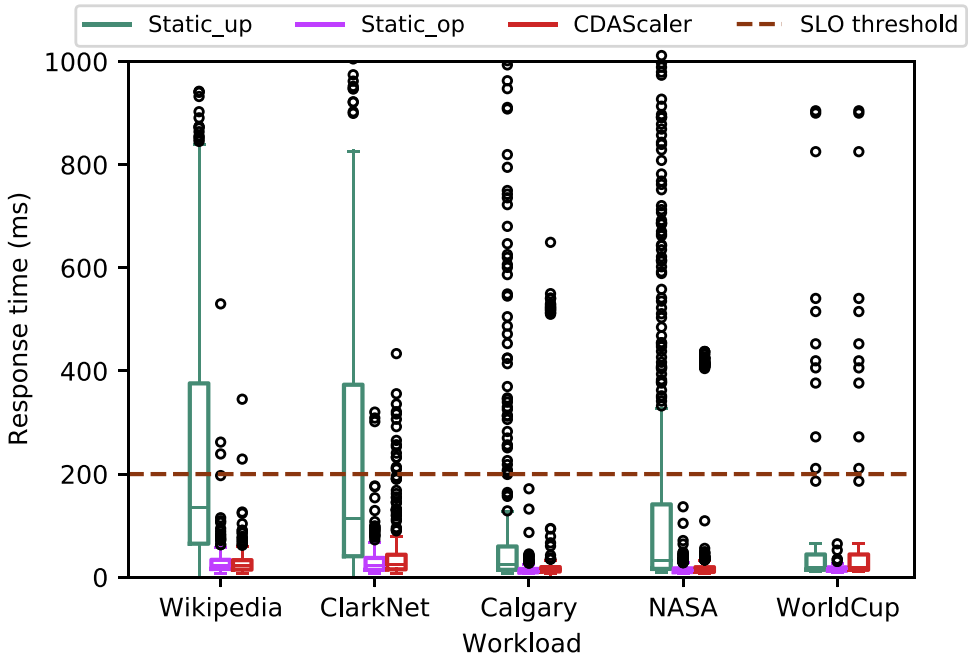


Fig. 9 Response time comparison between proposed, static_up, and static_op baseline methods

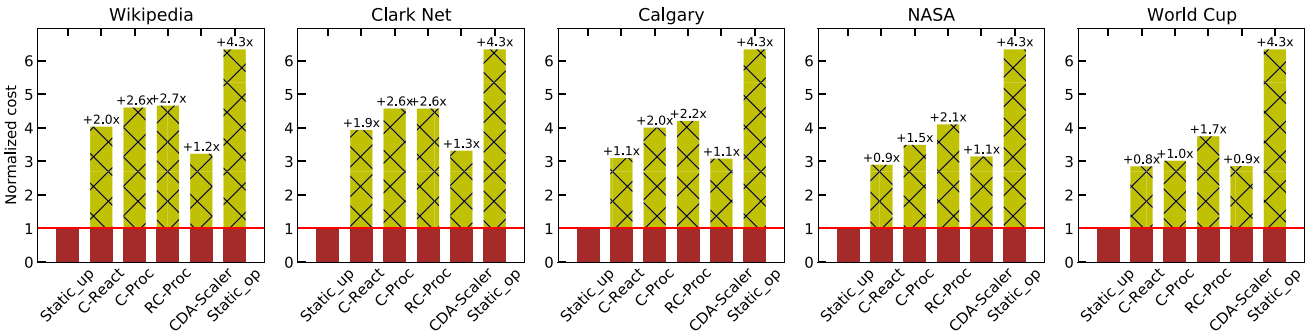


Fig. 10 Cost comparison between static_up, reactive, proactive, predictive, proposed, and static_op by satisfying the response time SLA

5.2 Discussion

The existing state-of-the-art autoscaling methods use horizontal scaling for containerized microservices by adding different numbers of containers with fixed or predefined CPU cores. However, the dynamic allocation of CPU cores for the containers during autoscaling is challenging. The proposed solution introduces an autoscaling method that considers the automatic CPU cores allocation for containers during the horizontal autoscaling for each microservice.

The extensive evaluation shows the effectiveness of our proposed method as compared to the baseline method using a benchmark microservices application under different real-time workloads. The experimental evaluation shows the proposed system outperforms by reducing the total number of unprocessed requests, SLA violations, scale operations, and cost compared to the existing state-of-the-art algorithms. The automatic CPU cores allocation during the autoscaling enables efficient resource utilization, leading to better performance and cost-effectiveness.

Overall, our proposed solution significantly contributes to the dynamic autoscaling of containerized microservices. It considers the automatic CPU cores allocation for each microservice apart from only allocating the number of replicas with fixed CPU cores, which leads to efficient resource utilization and better performance.

6 Conclusion and future work

Efficient allocation of resources, such as CPU cores to the individual replicas while scaling containerized microservices horizontally, is challenging and requires novel solutions. In this paper, we addressed this problem by proposing CDAScaler, which automatically identifies the appropriate CPU resources for each individual replica

required to spawn during the horizontal scaling of the microservices-based application.

CDAScaler offers a significant advantage in optimizing performance and minimizing operational costs compared to state-of-the-art solutions. The proposed solution outperformed the existing solution for maximizing processed requests while minimizing the unprocessed requests, SLA violations, cost, response time, and a number of scale operations. Additionally, we also compared CDAScaler with resource management approaches, including static over-provisioning and under-provisioning strategies. The proposed system can help to ensure cost-effective scalability for microservices applications effectively. Overall, CDAScaler is a comprehensive and efficient solution that enhances performance, reliability, and cost-effectiveness, demonstrating its potential over the traditional autoscaling approaches for microservice-based applications.

Currently, we are extending CDAScaler by incorporating other resource metrics, including memory, IO, and network bandwidth provisioning dynamically in the scaling decision for each container. Also, we plan to extend CDAScaler to optimize physical infrastructure utilization by improving container placement strategies. Moreover, we will test and evaluate the deep learning and reinforcement learning models for workload forecasting and resource provisioning.

Author Contributions Conceptualization: NS, MA & WI; methodology: NS, MA, WI, & AE; formal analysis and investigation: NS, MA, & WI; writing original draft preparation: NS, MA, WI,& FB; writing review and editing: MA, WI, FB, & AE; supervision: MA, WI, & FB

Funding No funding sources are applicable for this research

Data availability Not applicable

Declarations

Conflict of interest The authors declare that they have no conflict of interest

References

1. Vinoth, S., Vemula, H.L., Haralayya, B., Mamgain, P., Hasan, M.F., Naved, M.: Application of cloud computing in banking and e-commerce and related security threats. Mater. Today **51**, 2172–2175 (2022)
2. Dörnemann, T., Juhnke, E., Freisleben, B.: On-demand resource provisioning for bpel workflows using amazon's elastic compute cloud. In: 2009 9th IEEE/ACM International Symposium on Cluster Computing and the Grid, pp. 140–147 (2009). IEEE
3. Llorente, I.M., Moreno-Vozmediano, R., Montero, R.S.: Cloud computing for on-demand grid resource provisioning. Adv. Parallel Comput. **18**(5), 177–191 (2009)
4. Buyya, R., Garg, S.K., Calheiros, R.N.: Sla-oriented resource provisioning for cloud computing: Challenges, architecture, and solutions. In: 2011 International Conference on Cloud and Service Computing, pp. 1–10 (2011). IEEE
5. Villamizar, M., Garces, O., Ochoa, L., Castro, H., Salamanca, L., Verano, M., Casallas, R., Gil, S., Valencia, C., Zambrano, A., *et al.*: Infrastructure cost comparison of running web applications in the cloud using aws lambda and monolithic and microservice architectures. In: 2016 16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), pp. 179–182 (2016). IEEE
6. Acharya, J.N., Suthar, A.C.: Docker container orchestration management: a review. In: International Conference on Intelligent Vision and Computing, pp. 140–153 (2022). Springer
7. Amaral, M., Polo, J., Carrera, D., Mohomed, I., Unuvar, M., Steinder, M.: Performance evaluation of microservices architectures using containers. In: 2015 IEEE 14th International Symposium on Network Computing and Applications, pp. 27–34 (2015). IEEE
8. Mao, Y., Fu, Y., Gu, S., Vhaduri, S., Cheng, L., Liu, Q.: Resource management schemes for cloud-native platforms with computing containers of docker and kubernetes. arXiv preprint arXiv:2010.10350 (2020)
9. Abid, A., Manzoor, M.F., Farooq, M.S., Farooq, U., Hussain, M.: Challenges and issues of resource allocation techniques in cloud computing. KSII Trans. Internet Inf. Syst. **14**(7), 2815–2839 (2020)
10. Vinothina, V.V., Sridaran, R., Ganapathi, P.: A survey on resource allocation strategies in cloud computing. Int. J. Adv. Comput. Sci. Appl. (2012). https://doi.org/10.14569/IJACSA.2012.030616
11. Podolskiy, V., Jindal, A., Gerndt, M.: Iaas reactive autoscaling performance challenges. In: 2018 IEEE 11th International Conference on Cloud Computing (CLOUD), pp. 954–957 (2018). IEEE
12. Iqbal, W., Dailey, M.N., Carrera, D., Janecek, P.: Adaptive resource provisioning for read intensive multi-tier applications in the cloud. Future Gener. Comput. Syst. **27**(6), 871–879 (2011)
13. Roy, N., Dubey, A., Gokhale, A.: Efficient autoscaling in the cloud using predictive models for workload forecasting. In: 2011 IEEE 4th International Conference on Cloud Computing, pp. 500–507 (2011). IEEE
14. Imdoukh, M., Ahmad, I., Alfailakawi, M.G.: Machine learning-based auto-scaling for containerized applications. Neural Comput. Appl. **32**(13), 9745–9760 (2020)
15. Wajahat, M., Karve, A., Kochut, A., Gandhi, A.: Mlscale: a machine learning based application-agnostic autoscaler. Sustain. Comput. **22**, 287–299 (2019)
16. Abdullah, M., Iqbal, W., Mahmood, A., Bukhari, F., Erradi, A.: Predictive autoscaling of microservices hosted in fog microdata center. IEEE Syst. J. **15**(1), 1275–1286 (2020)
17. Abdullah, M., Iqbal, W., Berral, J.L., Polo, J., Carrera, D.: Burst-aware predictive autoscaling for containerized microservices. IEEE Trans. Serv. Comput. **15**(3), 1448–1460 (2020)
18. Cloud, G.: Microservice Demo Benchmark. https://github.com/GoogleCloudPlatform/microservices-demo
19. Thönes, J.: Microservices. IEEE Softw. **32**(1), 116–116 (2015)
20. Dragoni, N., Giallorenzo, S., Lafuente, A.L., Mazzara, M., Montesi, F., Mustafin, R., Safina, L.: Microservices: Yesterday, Today, and Tomorrow, pp. 195–216. Springer, New York (2017)
21. Usman, M., Ferlin, S., Brunstrom, A., Taheri, J.: A survey on observability of distributed edge & container-based microservices. IEEE Access **10**, 86904–86919 (2022)
22. Balalaie, A., Heydarnoori, A., Jamshidi, P.: Migrating to cloud-native architectures using microservices: an experience report. In:

European Conference on Service-Oriented and Cloud Computing, pp. 201–215 (2015). Springer
23. Shakarami, A., Shakarami, H., Ghobaei-Arani, M., Nikougoftar, E., Faraji-Mehmandar, M.: Resource provisioning in edge/fog computing: a comprehensive and systematic review. J. Syst. Architect. **122**, 102362 (2022)
24. Li, X., Pan, L., Liu, S.: A survey of resource provisioning problem in cloud brokers. J. Network Comput. Appl. **203**, 103384 (2022)
25. Khan, S.A., Abdullah, M., Iqbal, W., Butt, M.A., Bukhari, F., Hassan, S.-U.: Automatic migration-enabled dynamic resource management for containerized workload. IEEE Syst. J. **17**(2), 2378–2389 (2022)
26. Tseng, F.-H., Wang, X., Chou, L.-D., Chao, H.-C., Leung, V.C.: Dynamic resource prediction and allocation for cloud data center using the multiobjective genetic algorithm. IEEE Syst. J. **12**(2), 1688–1699 (2017)
27. Yadav, M.P., Yadav, D.K., et al.: Resource provisioning through machine learning in cloud services. Arab. J. Sci. Eng. **47**(2), 1–23 (2022)
28. Vayghan, L.A., Saied, M.A., Toeroe, M., Khendek, F.: Kubernetes as an availability manager for microservice applications. arXiv preprint arXiv:1901.04946 (2019)
29. Mostofi, V.M., Krul, E., Krishnamurthy, D., Arlitt, M.: Trace-driven scaling of microservice applications. IEEE Access **11**, 29360–29379 (2023)
30. Cai, B., Wang, B., Yang, M., Guo, Q.: Automan: resource-efficient provisioning with tail latency guarantees for microservices. Future Gener. Comput. Syst. **143**, 6–75 (2023)
31. Savitha, S., Sangana, C., Devendran, K., Pravin, L., Rajkumar, M., Nirmal, C.: Auto scaling infrastructure with monitoring tools using linux server on cloud. In: 2023 7th International Conference on Computing Methodologies and Communication (ICCMC), pp. 45–52 (2023). IEEE
32. Abdullah, M., Iqbal, W., Bukhari, F., Erradi, A.: Diminishing returns and deep learning for adaptive cpu resource allocation of containers. IEEE Trans. Network Serv. Manag. **17**(4), 2052–2063 (2020)
33. Rossi, F., Cardellini, V., Presti, F.L., Nardelli, M.: Dynamic multi-metric thresholds for scaling applications using reinforcement learning. IEEE Trans. Cloud Comput. **11**(2), 1807–1821 (2022)
34. Yu, G., Chen, P., Zheng, Z.: Microscaler: cost-effective scaling for microservice applications in the cloud with an online learning approach. IEEE Trans. Cloud Comput. **10**(2), 1100–1116 (2020)
35. Yan, M., Liang, X., Lu, Z., Wu, J., Zhang, W.: Hansel: adaptive horizontal scaling of microservices using bi-lstm. Appl. Soft Comput. **105**, 107216 (2021)
36. Hossen, M.R., Islam, M.A.: A lightweight workload-aware microservices autoscaling with qos assurance. arXiv preprint arXiv:2202.00057 (2022)
37. Ruíz, L.M., Pueyo, P.P., Mateo-Fornés, J., Mayoral, J.V., Tehàs, F.S.: Autoscaling pods on an on-premise kubernetes infrastructure qos-aware. IEEE Access **10**, 33083–33094 (2022)
38. Razavi, K., Luthra, M., Koldehofe, B., Mühlhäuser, M., Wang, L.: Fa2: Fast, accurate autoscaling for serving deep learning inference with sla guarantees. In: Proceedings of the 28th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS 2022) (2022). IEEE
39. Khaleq, A.A., Ra, I.: Intelligent autoscaling of microservices in the cloud for real-time applications. IEEE Access **9**, 35464–35476 (2021)
40. Sachidananda, V., Sivaraman, A.: Learned autoscaling for cloud microservices with multi-armed bandits. arXiv preprint arXiv:2112.14845 (2021)
41. Ezzeddine, M., Tauvel, S., Baude, F., Huer, F.: On the design of sla-aware and cost-efficient event driven microservices. In: Proceedings of the Seventh International Workshop on Container Technologies and Container Clouds, pp. 25–30 (2021)
42. Zafeiropoulos, A., Fotopoulou, E., Filinis, N., Papavassiliou, S.: Reinforcement learning-assisted autoscaling mechanisms for serverless computing platforms. Simul. Model. Pract. Theory **116**, 102461 (2022)
43. Toka, L., Dobreff, G., Fodor, B., Sonkoly, B.: Machine learning-based scaling management for kubernetes edge clusters. IEEE Trans. Netw. Serv. Manag. **18**(1), 958–972 (2021)
44. Dang-Quang, N.-M., Yoo, M.: Deep learning-based autoscaling using bidirectional long short-term memory for kubernetes. Appl. Sci. **11**(9), 3835 (2021)
45. Joyce, J.E., Sebastian, S.: Reinforcement learning based autoscaling for kafka-centric microservices in kubernetes. In: 2022 IEEE 4th PhD Colloquium on Emerging Domain Innovation and Technology for Society (PhD EDITS), pp. 1–2 (2022). IEEE
46. Cheng, L., Kalapgar, A., Jain, A., Wang, Y., Qin, Y., Li, Y., Liu, C.: Cost-aware real-time job scheduling for hybrid cloud using deep reinforcement learning. Neural Comput. Appl. **34**(21), 18579–18593 (2022)
47. Cheng, L., Wang, Y., Cheng, F., Liu, C., Zhao, Z., Wang, Y.: A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling. IEEE Trans. Sustain. Comput. (2023). https://doi.org/10.1109/TSUSC.2023.3303898
48. Xu, M., Song, C., Ilager, S., Gill, S.S., Zhao, J., Ye, K., Xu, C.: Coscal: multi-faceted scaling of microservices with reinforcement learning. IEEE Trans. Netw. Serv. Manag. **19**(4), 3995–4009 (2022)
49. James, G., Witten, D., Hastie, T., Tibshirani, R., Taylor, J.: Linear regression, 69–134 (2023)
50. Kim, I.K., Wang, W., Qi, Y., Humphrey, M.: Forecasting cloud application workloads with cloudinsight for predictive resource management. IEEE Trans. Cloud Comput. **10**(3), 1848–1863 (2022). https://doi.org/10.1109/TCC.2020.2998017
51. Arlitt, M., Jin, T.: A workload characterization study of the 1998 world cup web site. IEEE Netw. **14**(3), 30–37 (2000)
52. Urdaneta, G., Pierre, G., Van Steen, M.: Wikipedia workload analysis for decentralized hosting. Comput. Netw. **53**(11), 1830–1845 (2009)
53. Bangari, K., Rao, C.: Real workload characterization and synthetic workload generation. IJRET **5**(5), 417–429 (2016)
54. Kumar, J., Goomer, R., Singh, A.K.: Long short term memory recurrent neural network (lstm-rnn) based workload forecasting model for cloud datacenters. Procedia Comput. Sci. **125**, 676–682 (2018)
55. Ghobaei-Arani, M., Jabbehdari, S., Pourmina, M.A.: An autonomic resource provisioning approach for service-based cloud applications: a hybrid approach. Future Gener. Comput. Syst. **78**, 191–210 (2018)
56. Prachitmutita, I., Aittinonmongkol, W., Pojjanasuksakul, N., Supattatham, M., Padungweang, P.: Auto-scaling microservices on iaas under sla with cost-effective framework. In: 2018 Tenth International Conference on Advanced Computational Intelligence (ICACI), pp. 583–588 (2018). IEEE
57. Chieu, T.C., Mohindra, A., Karve, A.A., Segal, A.: Dynamic scaling of web applications in a virtualized cloud computing environment. In: IEEE International Conference on e-Business Engineering, pp. 281–286 (2009)
58. Ali-Eldin, A., Tordsson, J., Elmroth, E.: An adaptive hybrid elasticity controller for cloud infrastructures. In: 2012 IEEE Network Operations and Management Symposium, pp. 204–212 (2012)

59. Ali-Eldin, A., Kihl, M., Tordsson, J., Elmroth, E.: Efficient provisioning of bursty scientific workloads on the cloud using adaptive elasticity control, 31–40 (2012)
60. Iqbal, W., Dailey, M.N., Carrera, D., Janecek, P.: Adaptive resource provisioning for read intensive multi-tier applications in the cloud. Future Gener. Comput. Syst. **27**(6), 871–879 (2011)
61. Moreno-Vozmediano, R., Montero, R.S., Huedo, E., Llorente, I.M.: Efficient resource provisioning for elastic cloud services based on machine learning techniques. J. Cloud Comput. **8**(1), 5 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Numan Shafi is a Ph.D. student in the Faculty of Computing and Information Technology, University of the Punjab, Lahore, Pakistan. His research interests include in cloud computing, machine learning, and data science. He has published in several peer-reviewed journals. He is also a lecturer at the University of Engineering and Technology, Lahore, Pakistan.



Muhammad Abdullah is an Assistant Professor at the Faculty of Computing and Information Technology, University of the Punjab, Lahore, Pakistan. He received MPhil and Ph.D. in Computer Science degrees from the University of the Punjab, Lahore, Pakistan, in 2016 and 2020, respectively. His research interests include cloud computing, machine learning, scalable applications, and system performance management.



Waheed Iqbal is an Associate Professor at the Faculty of Computing and Information Technology, University of the Punjab, Lahore, Pakistan. He worked as a Postdoc researcher with the Department of Computer Science and Engineering, Qatar University during 2017–2018. His research interests lie in cloud computing, distributed systems, machine learning, and large scale system performance evaluation. Waheed received his Ph.D. degree from the Asian Institute of Technology, Thailand. He received dual Masters degrees in Computer Science and Information Technology from the Asian Institute of Technology and the Technical University of Catalonia (UPC), Barcelona, Spain, respectively.



Abdelkarim Erradi is an Associate Professor in the Computer Science and Engineering Department at Qatar University. His research and development activities and interests focus on autonomic computing, self-managing systems and cybersecurity. He leads several funded research projects in these areas. He has authored several scientific papers in international conferences and journals. He received his Ph.D. in computer science from the University of New South Wales, Sydney, Australia. Besides his academic experience, he possesses 12 years professional experience as a Designer and a Developer of large scale enterprise applications.



Faisal Bukhari is an Associate Professor at Punjab University College of Information Technology (PUCIT), University of the Punjab (PU), Lahore, Pakistan. He received the M.Sc. in Statistics and M.Sc. in Computer Science from PU. He got the M.S. and Ph.D. in Computer Science from the Asian Institute of Technology (AIT), Thailand. Currently, he is supervising Imaging and Data Science Lab at PUCIT. His research interests lie in computer vision, image processing, data science, and machine learning.